UNIVERSITÉ DE CAEN **Année universitaire 2025-2026**

IUT Grand Ouest Normandie **Département Réseaux et Télécommunications**

SAÉ 1.05 — Traiter des données — Cahier des charges du projet

Martin Pépin

1 Généralités

1.1 Organisation

- Le projet est à réaliser en **binômes**. Vous devez renseigner vos groupes sur le pad disponible sur la page Ecampus de la SAÉ, au **plus tard** une semaine après le début de la SAÉ. S'il y a un nombre impair d'étudiant·es, certain·es devront travailler en autonomie (trinômes interdits).

- Une personne du binôme fera un rendu sur Ecampus. Il sera constitué de votre code et d'un très court rapport (1 ou 2 pages, voir plus bas). Date de fermeture du dépôt : **vendredi 19 décembre 2025** à 23h59 pour les alternants et **vendredi 23 janvier 2026** à 23h59 pour les initiaux.
  Retard → pénalité.

- Votre code doit à minima s'exécuter correctement sur la version de Python installée en salle de TP (version 3.11 ? à vérifier).

- Votre code doit être rendu dans le format `.txt`. Bien évidemment, vos fichiers Python seront au format `.py` pendant toute la durée du projet. Je vous demande de les renommer au moment du rendu because Ecampus.

- Le projet est quasiment intégralement en autonomie. Cependant
  - vous disposez de documents « guides » au format TP pour vous aider à démarrer le projet ;
  - à tout moment vous pouvez poser des questions à vos enseignants (M. Brisacier et M. Pépin) si vous bloquez, n'attendez pas le dernier moment.

- Vous serez évalué à mi-projet et en fin de projet sur le travail accompli jusque là. Nous vérifierons que **vous comprenez** le code que vous avez écrit.
  J'en profite pour rappeler que le plagiat est une fraude qui est traitée en conseil de discipline. Utiliser des IA génératives pour faire le projet à votre place est donc doublement pénalisant puisque :
  1. vous serez mal évalué car vous ne comprendrez pas votre code ;
  2. vous prenez le risque de sanctions lourdes.

  Faites preuve de bon sens : le but de la SAÉ est d'évaluer **votre** capacité à coder en Python sur un petit projet. Je préfère un projet pas terminé faute de temps mais qui montre vos compétences et qui vous aura appris quelque chose à un projet fini avec ChatGPT qui fait perdre du temps à tout le monde.

1.2 Contenu du rapport

J'attends un **très court** rapport (1 à 2 pages max) au format **Markdown** avec les 3 sections suivantes :

1. Les choix que vous avez faits.
   → Choix sur les parties du sujet où je vous ai laissé de la liberté.
   → Choix techniques qu'il vous semble pertinent de mentionner.
2. Les difficultés que vous avez rencontrées et les éventuelles solutions que vous avez trouvées.
3. L'état d'avancement du projet au moment du rendu s'il n'est pas fini.
4.

1.3 Vue d'ensemble

Le but du projet est de travailler avec les différentes API du projet de cartographie collaborative OpenStreetMap [1], en Python, afin de récupérer quelques informations utiles et produire des statistiques (plus ou moins utiles).

Vous produirez des fiches aux formats Markdown et HTML, afin de présenter vos résultats de façon agréable à lire.

On veut réaliser les deux sous-projets suivants.

`fiche_osm`. Le but de ce projet est d'écrire un outil en Python permettant de produire une fiche de synthèse sur un point d'intérêt OSM, aux formats Markdown et HTML.

Cette fiche indiquera toutes les informations disponibles sur l'objet : son nom, ses horaires d'ouverture, ses coordonnées, le site web du lieu, le type de cuisine pour un restaurant, etc.

Notre objectif final est d'avoir script qui prend un identifiant OpenStreetMap en entrée et qui produit une page HTML.

`infos_locales`. Le but de ce projet est d'écrire un outil en Python permettant de récupérer tous les objets d'un certain type (par exemple tous les restaurants) dans une zone géographique donnée (par exemple la ville de Caen). Là aussi, vous produirez un résultat sous la forme d'une fiche aux formats Markdown et HTML.

Cette fiche affichera la liste de tous les objets trouvés dans la zone, et calculera quelques statistiques sur les objets. Des idées de statistiques sont indiquées plus bas.

2 Préliminaire : conversion Markdown → HTML

Dans les deux parties du projet, vous aurez besoin de convertir des fichiers markdown en HTML.

En vous aidant de la bibliothèque `markdown`, vous écrirez une fonction `convert` dans un fichier `md_to_html.py`, prenant en argument un nom de fichier Markdown et un nom de fichier HTML en convertissant le premier en le second. Écrire cette fonction dans un fichier à part vous permet de l'importer depuis les fichiers des deux autres projets :

1. https://www.openstreetmap.org/about

```python
from md_to_html import convert
```

De plus, vous ferez en sorte que votre fichier `md_to_html.py` puisse aussi être utilisé comme script pour convertir un fichier en ligne de commande. Par exemple (d'autres options sont possibles), on pourrais vouloir invoquer le script de la façon suivante : `python3 md_to_html.py input.md output.html`

2.1 Attentes

- La fonction `convert` peut être importée et utilisée depuis un autre module.
- Le script `md_to_html.py` est fonctionnel.
- [**bonus**] Pour un résultat plus « fini », vous pouvez donner une belle interface à votre script à l'aide du module `argparse` de la bibliothèque standard.

3 Fiche OSM d'un point d'intérêt

L'objectif de cette partie est d'avoir un script Python qui génère automatiquement un ficher HTML décrivant un point d'intérêt OSM.

Par exemple, on peut voir sur https://www.openstreetmap.org que le célèbre kebab « Délices Magic Beau Gosse » est le nœud d'identifiant 5553036408 dans la base OSM. Je veux donc pouvoir générer une fiche sur ce kebab :

- en exécutant la commande `python3 fiche_osm.py 5553036408` ;
- et/ou via l'appel de fonction `fiche_osm(5553036408)` depuis l'interpréteur, en ayant préalablement importé la fonction depuis votre fichier.

3.1 Fonctions à implémenter

J'attends de vous que vous implémentiez les fonctions suivantes :

`get_node(id: int) -> dict` . Cette fonction prend un identifiant de nœud OSM en argument et télécharge toutes les données actuelles sur ce nœud via l'API REST https://www.openstreetmap.org/api/0.6.

La documentation de cette API se situe à l'adresse https://wiki.openstreetmap.org/wiki/API_v0.6.

Vous devrez interroger l'API JSON et renvoyer un dictionnaire qui stocke toutes ces données.

`node_to_md(data: dict, filename: str) -> None` . Cette fonction prend un dictionnaire représentant un objet OSM et un nom de fichier, et génère un fichier markdown représentant toutes les informations sur l'objet OSM de façon structurée (titres, listes, etc).

Cette fonction ne renvoie rien.

`fiche_osm(id: int) -> None` . Cette fonction utilise les deux fonction précédentes, ainsi que la fonction `convert` que vous aurez implémentée dans le module `md_to_html`, pour générer automatiquement une fiche Markdown et une fiche HTML à partir d'un identifiant de nœud OSM.

Cette fonction de renvoie rien, mais pourra afficher des informations à l'utilisateurice, si ça vous semble utile, pour indiquer que tout s'est bien passé.

De plus, faites en sorte que votre fichier puisse être utilisé comme un script pour générer une page depuis le terminal. Par exemple, quelque chose comme `python3 fiche_osm.py 5553036408` devrait gérer les deux fichiers demandés.

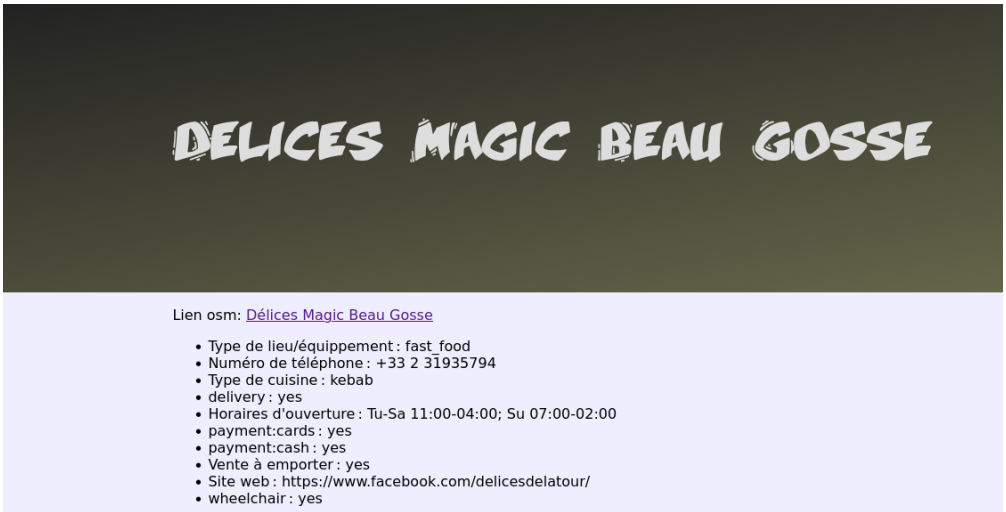
3.2 Format du résultat

Le format du fichier Markdown et du fichier HTML est laissé assez libre. L'essentiel est de présenter les données de façon pertinente. En particulier, utilisez les listes, titres, etc. à bon escient.

Il n'est pas non plus nécessaire de fournir une feuille de style avec le fichier HTML. Aucun point ne sera donné le CSS, ne faite du CSS que si vous avez terminé tout le reste.

En revanche, il est attendu que la page HTML soit du HTML valide. Regardez bien le résultat produit par la bibliothèque `markdown` et ajoutez ce qui manque.

3.3 Exemple de résultat possible



3.4 Attentes

— Vous faites correctement la requête sur l'API OSM.
— Les fonctions demandées sont toutes implémentées et fonctionnent.
— Le fichier peut être utilisé comme un script.
— Le résultat présente bien les données.

4 Récupération d'objets sur une zone et statistiques

La deuxième partie du projet consiste à dresser une liste, et faire des statistiques, sur l'ensemble des points d'intérêt d'un certain type sur une zone géographique donnée. À titre d'exemple, on peut s'intéresser à tous les restaurants sur la ville de Caen, à tous les cinémas en Normandie, à tous les hotels à Barcelone, etc. Dans mes exemples, je vais me concentrer sur les restos et fast-foods sur Caen et Ifs, mais vous faites (à peu près) ce que vous voulez.

Remarque importante : Il faut faire un choix, et un choix raisonnable.

Je vous laisse la liberté de choisir les types d'objets qui vous intéressent, et la région que vous souhaitez couvrir. Amusez vous, laissez parler votre imagination, faites quelque chose de rigolo ! Il y a beaucoup de choix sur OpenStreetMap, j'aimerais voir plein de projets différents. Voici quand même trois critères (de bon sens) à respecter.

- Il doit y avoir assez d'objets pour faire des statistiques : les cinémas sur Caen, c'est trop peu.
- Il ne doit pas y avoir trop d'objets, télécharger tous les restaurants d'Europe, c'est lourd, et c'est très malpoli vis à vis des autres utilisateurices de l'API.
- Les objets doivent avoir des propriétés sur lesquelles faire des stats : on peut faire des stats sur beaucoup de choses dans OSM (même les abri-bus !) mais ne choisissez pas les ronds-points dans le Vercors ou vous aurez trop peu de choses à dire…

N'hésitez pas à consulter vos enseignants pour vérifier que votre projet tient la route.

Pour réaliser cette partie, vous utiliserez l'API Overpass, beaucoup mieux adaptée pour récupérer beaucoup de données que l'API OSM de base. Voici quelques liens pour vous aider :

- La documentation de l'API :
  https://wiki.openstreetmap.org/wiki/Overpass_API
- La documentation du langage « Overpass QL » utilisé par l'API :
  https://wiki.openstreetmap.org/wiki/Overpass_API/Overpass_QL
- Une interface web permettant de jouer avec l'API :
  https://overpass-api.de/api/interpreter

Évidemment, c'est une bonne idée d'aller examiner manuellement les points d'intérêt qui vous intéressent sur le site web d'OpenStreetMap afin d'étudier les attributs des objets et de comprendre comment ils sont utilisés.

4.1 Fonctions à implémenter

J'attends de vous que vous implémentiez les fonctions suivantes :

`get_dataset(query: str) -> dict` . Cette fonction télécharge, au format JSON, toutes les données que vous avez choisi de récupérer dans le cadre de votre projet sous la forme d'un dictionnaire ou d'une liste (utilisation du module `json`).

La requête passée en argument doit être une requête Overpass QL.

`compute_statistics(...) -> ...` . Cette fonction calcule des statistiques « intéressantes » (voir plus bas) sur le jeu de données.

Je laisse les types des arguments et de la valeur de retour libre. Ça ne signifie pas qu'il ne faut rien mettre, ça signifie que je vous demander d'adapter la fonction à vos besoin.

ATTENTION !!!

Il est très important de bien documenter ce que fait votre fonction, je ne peux pas deviner sans la doc.

`dataset_to_md(dataset: dict, filename: str) -> None` . Cette fonction prend le jeu de donnée au format JSON téléchargé depuis l'API Overpass, et produit un fichier Markdown présentant :
- Les statistiques que vous avez calculées ;
- La liste de toutes les données.

Comme précédemment, soignez la présentation (titre, sous-titres, etc).

Cette fonction ne renvoie rien.

`infos_locales(...) -> None` . Cette fonction utilise toutes les fonctions précédentes pour télécharger un jeu de données et construire un fichier Markdown, et un fichier HTML présentant ces données et leur statistiques.

Cette fonction de renvoie rien, mais pourra afficher des informations à l'utilisateurice, si ça vous semble utile, pour indiquer que tout s'est bien passé.

4.2 Précisions sur les statistiques attendues

Je suis intéressé par des statistiques simples du type :
- Quelle proportions des fast-foods de Caen sont des kebabs/pizzerias/etc ?
- Quelle proportions des restaurants font de la vente à emporter ?
- Quelle proportions des magasins acceptent la carte bancaire ?
- À quelle point les abri-bus à Caen sont éclairés la nuit et/ou munis d'un banc et d'une poubelle ?
- Quels sont les 3 types de magasins les plus nombreux à Caen ?
- Quelle proportions des magasins ont leurs horaires d'ouverture renseignés dans OpenStreetMap ?
- À quel point les passages piétons à Caen sont-ils aménagés pour des personnes malvoyantes ?

Évidemment, vous n'avez pas besoin de me calculer tout ça, choisissez une, voir quelques, statistiques intéressantes en fonction de leur niveau de difficulté.

Notez que les statistiques du type « À quel point tel ou tel attribut est souvent renseigné dans OSM » me paraissent particulièrement intéressantes. OpenStreetMap est une base de donnée ouverte, n'importe qui peu contribuer et ce genre de statistiques indique à quel point la base est riche.

Là aussi, je vous laisse un peu de liberté, utilisez la à bon escient ! L'idée et de laisser parler votre imagination et votre créativité, pas de tomber dans la facilité et de simplement me dire qu'il y a 77 boulangeries à Caen.

4.3 Exemple de résultat possible

(Les statistiques ont été omises sur la capture.)

RESTAURANTS ET FAST-FOODS @ CAEN + IFS

INFORMATIONS LEGALES

Les données inclues dans ce document sont issues de www.openstreetmap.org et sont distribuées sous licence ODbL.

La police de caractères des titres est distribuée sous licence Arphic

INFORMATIONS TECHNIQUES

Date de récupération des données: 2023-11-10T19:33:14Z

Générateur de données: Overpass API 0.7.61.5 4133829e

Requête Overpass:

```
// Set a few options:
//   - we want json
//   - the request should not take more than 30s
[out:json][timeout:30];

// Define an area 'caen_and_ifs'
(
  area[wikipedia="fr:Caen"];
  area[wikipedia="fr:Ifs"];
)->.caen_and_ifs;

// Fetch some objets and store them in 'graille'
(
  nwr[amenity="fast_food"](area.caen_and_ifs);
  nwr[amenity="restaurant"](area.caen_and_ifs);
)->.graille;

// Output all the data in 'graille'
.graille out body;
```

LISTE DES LIEUX DE CONSOMMATION DE NOURRITURE

Chez Abi

- Type de lieu/équippement: fast_food
- Type de cuisine: kebab
- Horaires d'ouverture: Mo-Su 11:30-14:30,18:00-22:00

Pain Gourmand

- Type de lieu/équippement: fast_food

Le Félini

- Type de lieu/équippement: fast_food
- created_by: JOSM

Mon cher riz

- Type de lieu/équippement: restaurant

Fleur de Sushi

- Type de lieu/équippement: restaurant
- Type de cuisine: japanese
- email: fleurdesushi@orange.fr
- Horaires d'ouverture: Tu-Sa 12:00-15:00, 19:00-23:00
- phone: +33 231742664
- Site web: https://www.fleurdesushi.fr/
- wheelchair: no

Le Carlotta

4.4 Attentes

— Vous faites correctement vos requêtes sur l'API Overpass.
— Les fonctions demandées sont toutes implémentées et fonctionnent.
— Vous avez fait l'effort de calculer quelques stats intéressantes.
— Vous avez documenté vos fonction, et en particulier la fonction qui calcule des statistiques.
— Le fichier peut être utilisé comme un script (`python3 stats.py` voire plus compliqué si vos fonctions sont paramétrables).
— Le résultat présente bien les données.

5 Fonds de cartes [Initiaux uniquement]

Enfin, pour améliorer notre premier projet `fiche_osm`, nous allons pouvoir y ajouter une minicarte, grâce à une API de « tuiles ». C'est une API qui permet de télécharger des bouts de carte sous forme d'image, souvent au format PNG, à partir des données d'OSM.

5.1 Calcul des coordonnées d'une tuile

Pour ajouter une tuile à une fiche OSM, vous devez d'abord choisir un niveau de zoom, puis calculer les identifiants `x` et `y` de votre tuile à partir des coordonnées (latitude / longitude) du point qui vous intéresse.

Vous aurez besoin des formules fournies dans la documentation à l'adresse suivante : https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames. Je ne vous demande pas de comprendre les mathématiques derrière, vous pouvez juste utilisez la formule.

Par exemple, le kebab « Magic Beau Gosse » se trouve aux coordonnées `49.18444/-0.35930`. En utilisant la formule de la documentation, et notamment la fonction Python qui y est fournie, je trouve que ça correspond à `x=16351` et `y=11227` pour le niveau de zoom 15.

5.2 API de récupération des tuiles

Il existe différentes API pour télécharger des tuiles, avec différentes conditions d'utilisation, et différents styles graphiques. Voici une liste d'API fournissant ce services, à vous de choisir celui que vous préférez.

Voici par exemple, la tuile de zoom 15 contenant le kebab « Magic Beau Gosse » selon deux APIs différentes :

https://tile.openstreetmap.org/15/16351/11227.jpg

https://watercolormaps.collection.cooperhewitt.org/tile/watercolor/15/16351/11227.jpg

5.3 Manipulation des images

Afin de manipuler les images, en particulier si vous voulez assembler plusieurs tuiles ensemble pour faire une plus grande carte, ou si vous voulez ajouter un marqueur sur l'image aux coordonnées de l'objet, je vous suggère d'utiliser la bibliothèque `Pillow`.

Voici une image réalisée à l'aide de `Pillow` pour la fiche OSM du kebab « Magic Beau Gosse ».

5.4 Attentes

— Ajouter une fonction `generate_map` dans le fichier `fiche_osm` qui télécharge la (ou les) tuiles au niveau du point en question, et optionnellement ajouter un marqueur sur la carte.
— Intégrer la gestion de la carte dans `node_to_html` et `fiche_osm`.